0211. Design Add and Search Words Data Structure

Category: Tries | LeetCode: https://leetcode.com/problems/design-add-and-search

Problem

- AddWord(word) and search(pattern) where '.' matches any single letter.

Intuition

- Store words in a trie. For a normal letter follow one child; for '.' try every child (DFS/backtracking).

Brute Force

- Store a list and regex-match every word: O(N L) per search.

Optimal Approach

- AddWord: standard trie insert.
- Dfs(node, i): if i == len return node.end.
- If pattern[i] == '.', return true if any child c has dfs(c, i+1). Else follow the matching child.

Worked Example

```
add bad, dad, mad
search "pad" -> no 'p' child -> false
search "bad" -> true
search ".ad" -> try b: a,d end -> true
search "b.." -> b -> a -> d end -> true
```

Java Solution

```java
public class WordDictionary {
    private static class Node {
        Node[] next = new Node[26];
        boolean end;
    }

    private final Node root = new Node();

    public void addWord(String word) {
        Node n = root;
        for (char c : word.toCharArray()) {
            if (n.next[c - 'a'] == null) n.next[c - 'a'] = new Node();
            n = n.next[c - 'a'];
        }
        n.end = true;
    }

    public boolean search(String word) { return dfs(root, word, 0); }

    private boolean dfs(Node n, String w, int i) {
        if (i == w.length()) return n.end;
        char c = w.charAt(i);
        if (c == '.') {
            for (Node child : n.next) if (child != null && dfs(child, w, i + 1)) return true;
            return false;
        }
        Node child = n.next[c - 'a'];
        return child != null && dfs(child, w, i + 1);
    }
}
```

Complexity

- Add O(L). search O(L) without dots, O(26^d L) worst case with d dots.

Pitfalls

- A pattern longer than every word must return false, not throw.